

Hardening npm Publishing

サプライチェーン侵害から公開フローを守る

azu (@azu_re)

2026-06-23 / OSS開発者は今何をすべきか？

自己紹介

- Name : azu
- X/Twitter : [@azu_re](#)
- Website : [Web scratch](#), [JSer.info](#)
- Open Source : textlint / secretlint / HonKit など
- npmに公開しているパッケージは数百個



なぜ今、公開フローを守るのか

- npmは依存が深く、1つのパッケージ侵害が広い範囲に波及する
- 攻撃の起点がメンテナのトークンやCI/CDに移ってきている
- OIDCトークンの窃取、不正なworkflow、infostealerが実際に起きている
- 自分のパッケージが踏み台になる前提で守る

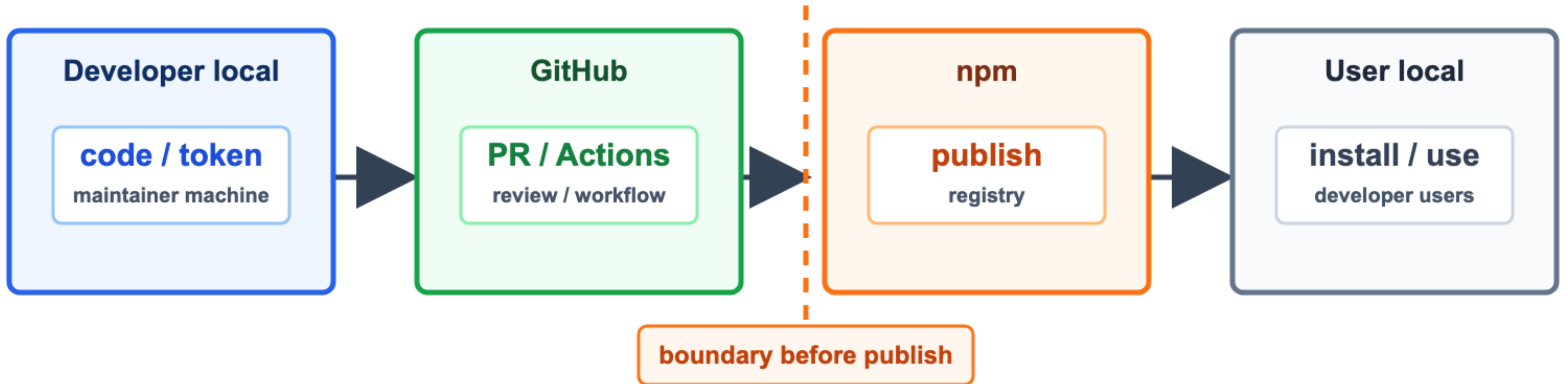
この発表の目的

- 改ざんをゼロにする話ではない
- 侵害されても、悪いpackageがregistryへ出る前に止める
- コードの中身を保証する話は主題にしない

問い: 公開フローのどこを守るか

- 攻撃者は、1箇所のインジェクトで全部抜けるなら一番弱い部分を狙う
- 1つの対策だけで公開フロー全体は守れない
- ローカルから公開までの経路を段階ごとに制御する
- どこかが破られても、別の段階で検出・制限する

Developer local → GitHub → npm → User local



公開までのレイヤー

1. ローカルのToken管理
2. トークンレスnpm (OIDC Trusted Publishing)
3. GitHub Actions (PR / Environments / Deployment protection)
4. npm staged publishing

1. ローカルのToken管理

生のcredentialをローカルに置かない

- 第一前提は、ローカルを抜かれても公開権限が漏れないこと
- ローカルファイルにcredentialを保存しない
- 1Password/Bitwardenなど多要素認証をしないと取り出せないところへ保存する
- 強いトークンをローカルに常駐させない

GitHub: classic PATを常用しない

- classic PAT(Personal Access Token)は強すぎる権限
- 使うとしてもローカル限定。CIでは使わない
- 常用はfine-grained PATにして、リポジトリと権限を絞る
- fine-grainedはリソースオーナーに紐づくので多少手間だが許容する

fine-grained PATは用途ごとに分ける

- 1つの強いトークンを使い回さず、用途ごとに発行する
- アクセスレベルを最小にする（read-only / 必要な操作だけ）
- 対象を絞る（特定リポジトリ / public のみ）
- 漏れても影響範囲がそのトークンの用途に限定される



read-onlyのトークンしか発行していない

課題: Checks APIがfine-grained未対応

- Checks APIがfine-grained PATに対応していない
- これが直れば、常用トークンから強いclassicをほぼ消せる

参考: github.com/orgs/community/discussions/129512

npm: アクセストークンを0個にする

Access Tokens					Generate New Token
☒ Name	Bypass 2FA	Created	Last used	Expires	
Rows 1 to 0 of 0					
 Read the Documentation					

2. トークンレス npm

OIDC Trusted Publishing

OIDC Trusted Publishingとは

- 長期トークンをやめ、short-livedでworkflow固有の署名トークンで公開する
- npmとGitHub ActionsがOIDCで信頼関係を結ぶ
- 「特定リポジトリの特定workflowからの実行」をnpmが確認できる
- npm 11.5.1以上が必要

参考: efcl.info/2025/09/07/npm-oidc/

npmjs.comでTrusted Publisherを登録

Publisher*

Organization or user*

Repository*

Workflow filename*

Filename only (e.g., publish.yml). Must exist in ``.github/workflows/`` in your repository.

Environment name

The name of the GitHub Actions environment used for publishing. This is encouraged for projects with maintainers who should not have npm publishing access.

Require 2FA and disallow tokens

Publishing access

Requiring an additional authentication method adds another level of security for your package.

- ☐ Require two-factor authentication or a granular access token with bypass 2fa enabled
- ☒ Require two-factor authentication and disallow tokens (recommended)

Note about trusted publishers: All publishing access options above are compatible with OIDC trusted publishers. If you have configured trusted publishers for this package, they will continue to work regardless of which option you select. For maximum security, we recommend using trusted publishers with the most restrictive token option.

Update Package Settings

workflow側の最小構成

permissions:

contents: write

id-token: write # OIDC

steps:

- uses: actions/setup-node@v5

with:

registry-url: 'https://registry.npmjs.org'

- run: npm publish --access public

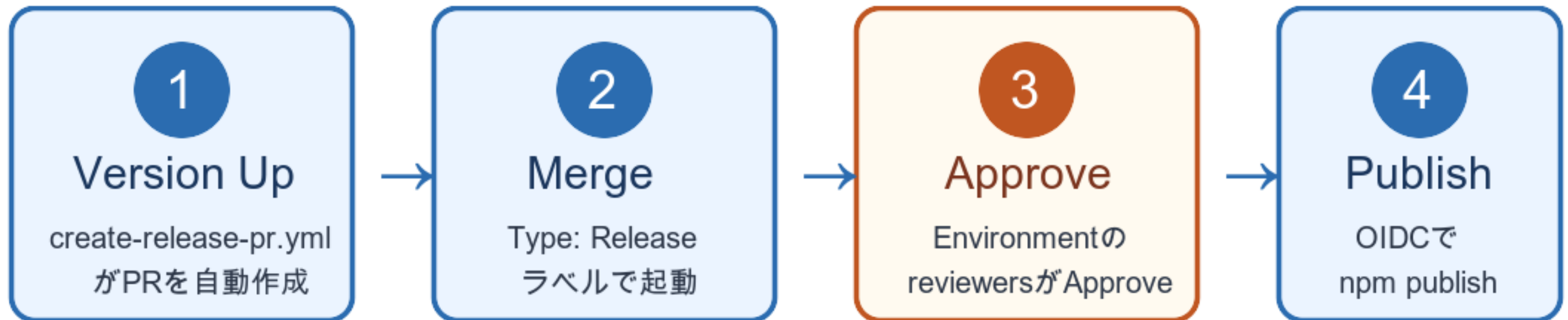
provenanceで来歴を残す

- OIDC公開ではprovenance（来歴の署名）が自動付与される
- どのリポジトリのどのworkflowでビルドされたかを証明できる
- npm provenanceはpackageとworkflowを結びつける証拠になる
- ただしbuild中に何が起きたかまでは保証しない

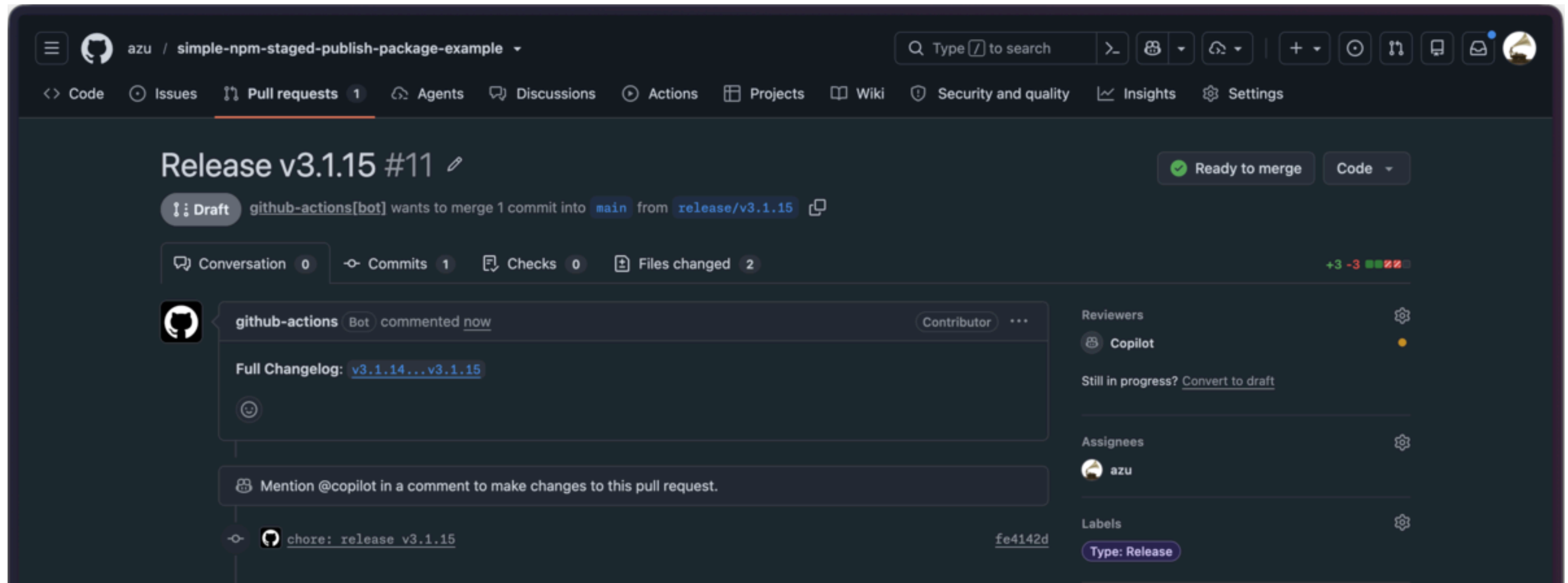
3. GitHub Actions の構成

Rulesets / Environments / Deployment
protection

Release PRの流れ



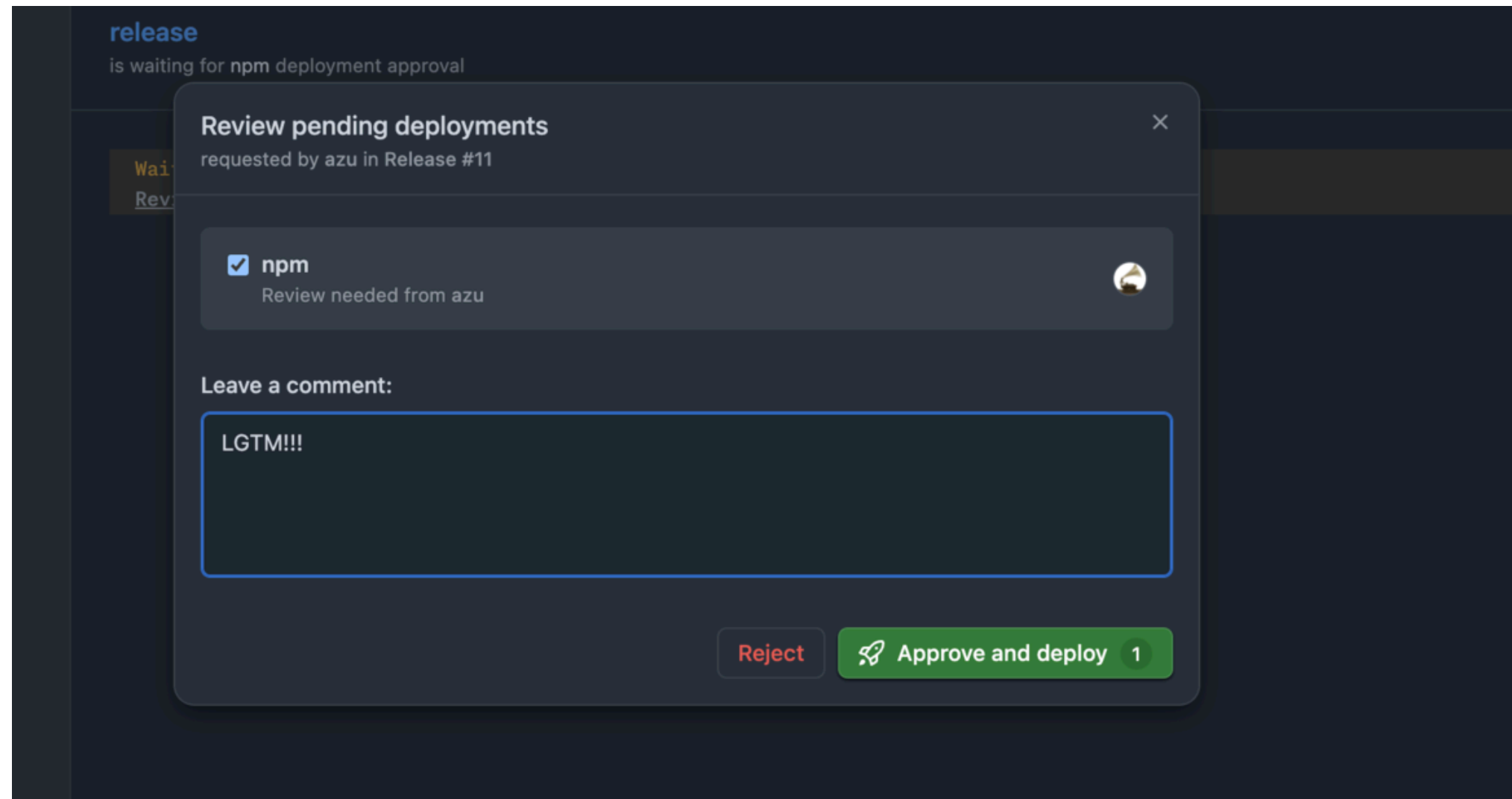
Step 1: Version Up PR



Step 2: Merge Release PR

- Release PRをレビューしてマージする
- mainに入ったrelease commitだけがpublish候補になる
- まだnpm publishは走らない
- 次のApproveで初めてrelease jobを進める

Step 3: Approve Deployment



Step 4: Publish to npm

- Approve後にrelease jobが実行する
- OIDCでnpmとtoken exchangeする
- npm publishまたはnpm stage publishを実行する
- provenance付きでregistryへ公開される

provenanceだけでは足りない

- provenanceで「どこから出たか」は分かる
- でも「作る途中で何が混ざったか」は分からない
- 悪いpackageにも正しい署名が付くことがある
- だからpublishに進む前に別のApproveを求める

参考: [Mini Shai-Hulud: Where SLSA's Boundaries Fall](#)

事例: OIDCだけでは止まらない

- workflow改変でOIDC credentialを取得
- credentialをログへ出して持ち出す
- GitHub write権限がnpm publish権限へ広がる
- 境界にEnvironment Approveを置く

参考: [GMO Flatt Security Blog](#)

EnvironmentでApproveとrefを制御する

Deployment protection rules

Configure reviewers, timers, and custom rules that must pass before deployments to this environment can proceed.

☒ **Required reviewers**
Specify people or teams that may approve workflow runs when they access this environment.

Add up to 5 more reviewers

Search for people or teams...

azu

×

☐ **Prevent self-review**
Require a different approver than the user who triggered the workflow run.

☐ **Allow administrators to bypass configured protection rules**

Deployment branches and tags

Limit which branches and tags can deploy to this environment based on rules or naming patterns.

Selected branches and tags ▾

0 branches and 0 tags allowed

refs/pull/*/merge

Currently applies to pull requests

Edit

Remove

+

 Add deployment branch or tag rule

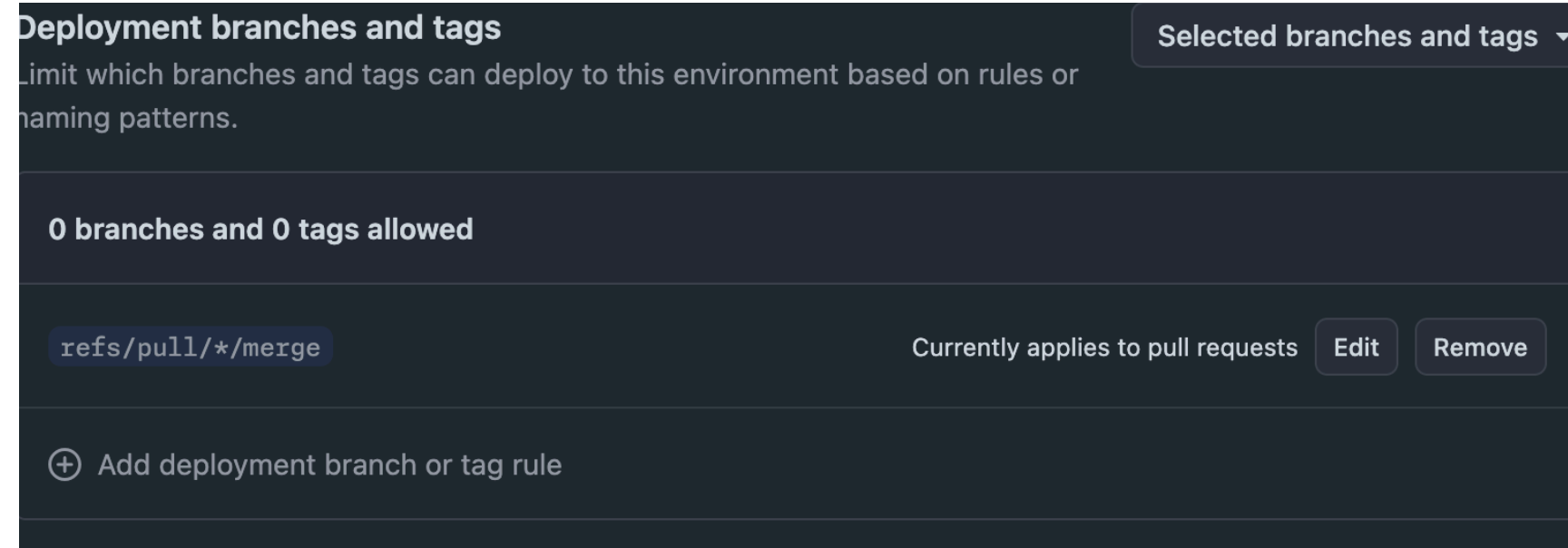
OSS開発者は今何をすべきか？ソフトウェアサプライチェーン侵害対策を考える - azu

28

merge refだけを許可する

- GitHubがPR用のmerge refを作る
- 許可するrefは `refs/pull/*/merge` のみ
- Approveまでrelease jobを進めない

参考: [GitHub Docs](#) / [GitHub Changelog](#)



改変だけではpublishへ進めない

- npmのTrusted Publisherはworkflowファイル名とEnvironment名を確認する
- Environmentは `refs/pull/*/merge` だけ許可する
- Environmentはrequired reviewersのApproveも必要
- workflowファイル名一致だけではOIDC交換まで進めない

Environment名をnpm側と一致させる

environment: npm

GitHub Actions / release.yml

```
11: |
  github.event.pull_request.merged == true &&
  contains(github.event.pull_request.labels.*.name, 'Type: Release')
  environment:
    name: npm
  permissions:
    contents: write
    id-token: write # OIDC
  outputs:
    released: ${ steps.tag-check.outputs.exists == 'false' }}
    version: ${ steps.package.outputs.version }}
    package-name: ${ steps.package.outputs.name }}
    release-url: ${ steps.create-release.outputs.url }}
  steps:
    - name: Checkout
      uses: actions/checkout@08c6903cd8c0fde910a37f88322edcfb5dd907a8 # v5.0.0
      with:
        persist-credentials: false
    - name: Get package info
```

npm

Workflow filename*

Filename only (e.g., publish.yml). Must exist in `.github/workflows/` in your repository.

release.yml

Environment name

The name of the [GitHub Actions environment](#) used for publishing. This is encouraged for projects with maintainers who should not have npm publishing access.

npm

外部Actionに依存しない

- 使うのは公式の checkout / setup-node のみ
- setup-nodeのcacheも使わない
- PRや低権限workflowの生成物をrelease workflowへ持ち込まない
- リポジトリ操作は gh / git コマンドで行う
- uses: はcommit SHAでpinする
- checkoutは persist-credentials: false

4. npm staged publishing

公開前にApprove stepを追加する

- npm publish は直接公開、npm stage publish はApprove待ちにする
- OIDC設定で publish と stage publish のどちらを許可するか選べる
- stage publishのみ許可にすると、直接公開は拒否される
- npm 11.15.0以上 / Node 22.14.0以上、OIDC時のみ利用可

参考: docs.npmjs.com/staged-publishing

staged publishingのフロー



Staged PackagesのApprove画面

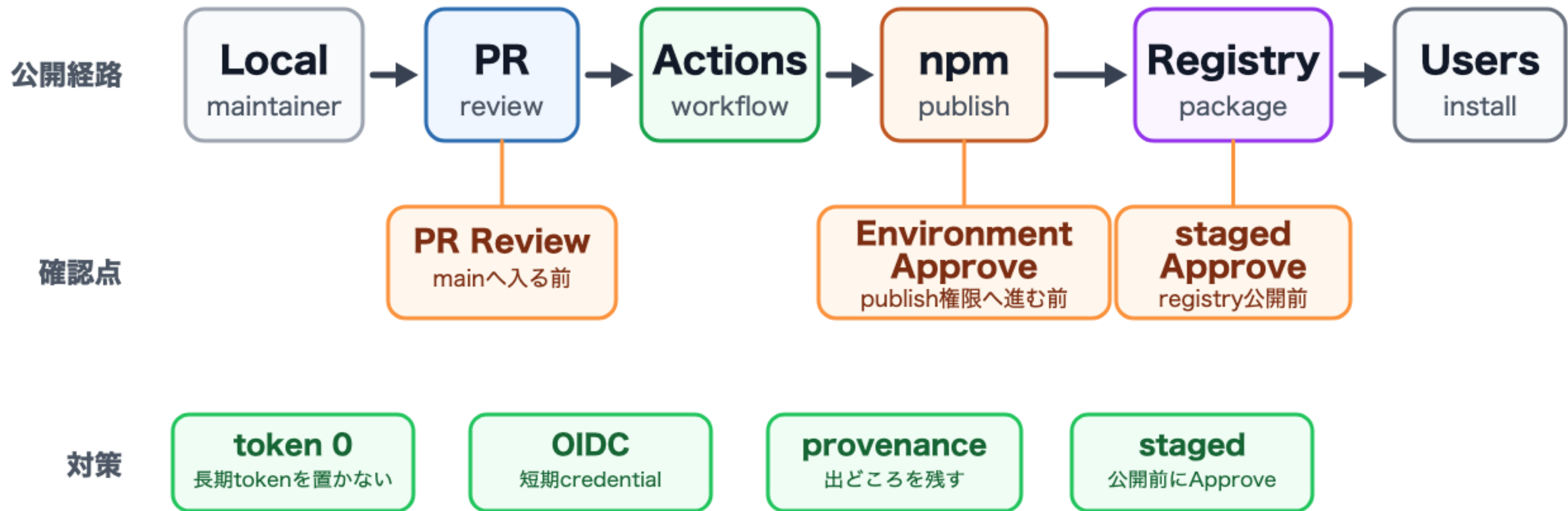
 [@azu/simple-npm-staged-publish-package-example](#) PUBLIC 3.1.15 latest

DATE	SHASUM	TRUSTED PUBLISHER
Jun 19, 2026, 7:04 AM UTC	035f14993fd42f6d2f6cf2536242d4d3eb4022b5	OIDC GitHub Actions

✓ Approve ✗ Reject ↓ Inspect

staged publishingの課題

- Approveが1パッケージずつしかできない
- monorepo（40個一括）だと現実的でない
- CLIからapproveするにはnpmトークンが要る
- npm / GitHubのどちらか一方はtokenlessに寄せる



まとめ

公開フローを段階ごとに制御する

1. ローカル: 強い権限を常駐させない
2. OIDC: npm tokenを持たない
3. Actions: PR + Environment + Approve
4. 権限境界: publish権限へ進む前にApprove
5. staged publishing: registry公開前にApprove

AIエージェント時代も同じ

- 全権限を1つの主体に集めない
- AIが全部の権限を持っているなら、攻撃者はAIを狙うだけ
- 最小権限と権限分離はやる必要がある

参考リンク

- npm OIDC: efcl.info/2025/09/07/npm-oidc/
- 公開例: github.com/azu/simple-oidc-example-package
- staged例: github.com/azu/simple-npm-staged-publish-package-example
- Bitwarden CLI侵害: [GMO Flatt Security Blog](#)
- TanStack侵害(cache poisoning): tanstack.com/blog/npm-supply-chain-compromise-postmortem
- Mini Shai-Hulud(SLSAの境界): slsa.dev/blog/2026/05/mini-shai-hulud-what-slsa-can-and-cannot-do
- npm staged publishing: docs.npmjs.com/staged-publishing
- GitHub changelog: [pull_request_target_and_environment_branch_protections](#)

ありがとうございました